

Real-Time Database Application Design Issues

Real-Time Systems

Reference: Rajib Mall, *Real-Time Systems: Theory and Practice*, Pearson Education India, 2007

Chapter: Real-Time Databases (IIT Kharagpur, Version 2 CSE)

Outline

- Motivation: Why real-time databases?
- Example applications of real-time databases
- Review of basic database concepts (RDBMS, transactions, ACID)
- Real-time databases: how they differ from traditional databases
- Temporal data and temporal consistency
- **Real-time database application design issues** (core topic)
- Concurrency control (brief)
- Summary and conclusion

Speaker note: State clearly that the heart of the talk is the “design issues” — everything before it builds the vocabulary needed to understand why the design is hard.

1. Motivation — Why Do We Need Real-Time Databases?

- Many real-time applications must **store large volumes of data** and **process them** for correct operation.
- Such storage needs arise when a controlling system must maintain an **up-to-date state** of a controlled system.
- Examples where large data must be stored and processed:
 - Network management systems
 - Industrial control systems
 - Autopilot / spacecraft control systems
- Whenever large amounts of data must be stored and processed, a **Database Management System (DBMS)** is used.
***Key idea:** A **Real-Time DBMS (RTDBMS)** is a DBMS used in data-intensive real-time applications where timing matters as much as correctness.*

2. Traditional vs. Real-Time Databases — First Look

Like traditional databases, real-time databases:

- Serve as **repositories of large volumes of data**
- Provide **efficient storage, retrieval, and manipulation** of data

But there are crucial differences. They differ mainly in:

- ① **Temporal characteristics** of the stored data
- ② **Timing constraints** imposed on database operations
- ③ **Performance goals**

These three differences are what make designing a satisfactory real-time database application much harder than a traditional one.

3. Example Applications of Real-Time Databases

A cross-section of applications with **stringent timing requirements**:

- **Process Control** — sensors/actuators; control decisions from input data + config parameters; transactions must complete in **a few milliseconds**, even under worst-case load.
- **Internet Service Management (SMS)** — ISPs manage e-mail, VPN, LDAP, etc.; RTDBMS handles **authorization, authentication, accounting** for *millions* of subscribers.
- **Spacecraft Control System** — monitors spacecraft “health” and trajectory; small data (a few MB) but **very stringent** timing and reliability (redundancy in HW/SW).
- **Network Management System** — stores topology, configuration, traffic, and fault data; many real-time transactions.

3. Applications — The Common Thread

Timing requirements span a wide range:

Application	Timing Order
Network routing decisions	microseconds
Opening/closing valves (process control)	milliseconds
Materials movement on factory floor	seconds

Regardless of the magnitude of the timing constraint: Unless the **transaction timing constraints** are met, the system fails.

4. Review of Basic Database Concepts

- A **relational database** = a set of *fact tables*; each table = several *records*.
- **Consistency constraints**: assertions restricting the values (or combinations of values) that records may assume.
- **Transaction**: a sequence of *reads* and *writes* that performs a high-level function and takes the database from **one consistent state to another**.
- Transactions usually run in an **interleaved** manner to improve **throughput** and **resource utilization**.
- **Goal**: maximize the number of transactions active at a time.
- A **schedule** = a particular sequencing of actions of different transactions.

Problem: concurrent execution can produce schedules that violate database integrity → we need concurrency control.

4. ACID Properties

Concurrency control maintains integrity by enforcing four properties:

- **Atomicity** — all or none of a transaction's operations are performed (single indivisible unit).
- **Consistency** — a transaction preserves the integrity constraints of the database.
- **Isolation** — concurrent transactions do not interfere with each other's computations.
- **Durability** — changes of a committed transaction are **permanent**, surviving later failures.

How they are ensured: - **Rollback protocols** → atomicity + durability when a transaction fails. - **Locking + rollback** → isolation.

4. Rollbacks and Cascading Aborts

- If transaction T_i reads a value written by an **aborted** transaction T_j , then T_i must also be aborted (to enforce atomicity).
- Rollbacks can trigger **cascaded aborts**.
- **Durability rule:** once a transaction commits, it cannot be aborted or reversed by cascading aborts.
- **Cascadeless aborts** are achieved by ensuring every transaction reads **only committed data**.
*Real-time implication: rollbacks undo significant completed work $\rightarrow$ the delay in redoing it can make a transaction **miss its deadline**.*

This is a preview of why real-time design is hard.

5. Real-Time Databases — The Three Key Differences

A real-time database differs from a traditional database on **three counts**:

- ① **Timing constraints** are associated with database operations.
- ② It must deal with **temporal (perishable) data**, not just static data.
- ③ The **performance metrics** that matter are very different.

We now elaborate each of these three issues.

5. Difference 1 — Temporal Data

Temporal (perishable) data: data whose validity is lost after a **prespecified time interval**.

Examples:

- **Temperature sensor:** transmits samples every ~100 ms; new readings make old data **stale** / **archival**.
- **Stock market quotes:** new quotations make previous ones **obsolete**.
- **Fly-by-wire aircraft:** current altitude, velocity, acceleration are *temporal*; the predetermined path is *non-temporal (archival)*.
*A real-time database must handle **both** temporal **and** archival data — and often combines them to derive new data.*

5. Difference 2 — Timing Constraints on Operations

- **Tasks** and **transactions** are similar: both are units of *work* and *scheduling*.
- But, unlike real-time tasks:
 - A transaction may need **many data records in exclusive mode**.
 - Task execution times can be assumed **deterministic**; transaction execution times are **much more unpredictable** — especially when **disk accesses** are required.

Unpredictability is the enemy of real-time guarantees.

5. Difference 3 — Performance Metric

- Common metric for **all** databases: **transaction response time**.
- **Traditional databases:** optimize **average** response time $\rightarrow$ *transactions completed per unit time* (throughput).
- **Real-time databases:** the metric of interest is the $>$ **number of transactions missing their deadlines per unit time**.

Traditional DBs optimize the average; real-time DBs care about deadline misses.

6. CORE TOPIC — Real-Time Database Application Design Issues

Design of a real-time database application is **much more intricate** than for non-real-time applications.

Why is it so hard? Let us investigate the reasons.

Speaker note: This slide series (Sec. 6) is the focus of the presentation.

6. Design Issue 1 — Data Access Delay

- Regardless of real-time or not, transactions have **extensive data requirements**.
- If data is stored in **secondary storage** (disk), the **access delay** can make a transaction **miss its deadline**.

***Consequence:** slow storage access directly threatens deadline guarantees.*

6. Design Issue 2 — Unpredictable Response Time

- It becomes **almost impossible to predict** transaction response time because of intricate protocols used to keep the database consistent:

- **Concurrency control protocols**
- **Commit protocols**
- **Recovery protocols**

*These protocols are essential for correctness, but they inject **unpredictable delays** — the opposite of what real-time systems need.*

6. Design Issue 3 — Cascading Rollbacks

- **Roll backs can have cascading effects.**
- They introduce **unpredictable amounts of delay.**
- Undoing and redoing accomplished work can push a transaction **past its deadline.**
***Summary of the problem:** Disk-access delay + protocol-induced unpredictability + cascading rollbacks $\Rightarrow$ hard to guarantee deadlines in a real-time database.*

6. Design Issues — “Silver Linings” (Solutions)

At first it may appear databases are **impractical for hard real-time** applications. But several **silver linings** exist:

- **In-memory databases:** keeping data in main memory makes many delay problems **vanish** (no disk-access delay).
- **Known, simple transaction set:** in real-time applications, transactions are **simple and known beforehand** (e.g., periodic sensor updates).
- **Fixed resource usage:** these transactions use the **same amount and types of data** each time.
 - $\Rightarrow$ Plans for **effective resource usage** can be made to achieve **deterministic transaction execution**.

Takeaway: *real-time databases are feasible when we exploit predictability of the workload.*

7. Characteristics of Temporal Data (Supporting Concept)

- A real-time system = **controlled system (environment)** + **controlling system (computer)**.
- The controller maintains an **image of the environment** via **periodic polling** of sensor data.
- The environment changes **unpredictably** $\Rightarrow$ the stored state is **highly perishable**.
- Example — **antimissile system**: controller tracks a missile's position, velocity, acceleration; stored state must match the **actual** state.
*This need to match actual vs. perceived state leads to **temporal consistency**.*

7. Temporal Consistency — Two Requirements

Temporal consistency: the actual environment state and the state stored in the database must be **very close** (within application limits).

Two requirements:

- **Absolute Validity** — consistency between the *environment* and its *reflection in the database*.
- **Relative Consistency** — consistency **among data items** used together to derive new data.

7. Representing a Data Item

A data item is a **triplet**:

$$d = (\text{value}, \text{avi}, \text{timestamp})$$

- d_{value} — the recorded value of d
- $d_{\text{timestamp}}$ — time when d was measured
- d_{avi} — *absolute validity interval*: the time after the timestamp during which d is considered valid

Example: $d = (120, 5 \text{ msec}, 100 \text{ msec}) \rightarrow$ value 120, recorded at 100 msec, valid for 5 msec.

7. Validity & Consistency Conditions

Absolute Validity: d is absolutely valid iff
 $(\text{Current time} - d_{\text{timestamp}}) \leq d_{\text{avi}}$

Relative Consistency: a set R is relatively consistent iff for all $d, d' \in R$
 $|d_{\text{timestamp}} - d'_{\text{timestamp}}| \leq R_{\text{rvi}}$

where R_{rvi} is the *relative validity interval* of the relative-consistency set R .

7. Worked Examples

Ex 1: $d = (10, 2500 \text{ msec}, 100 \text{ msec})$, current time = 2700 msec. - Valid window: 2500 $\rightarrow$ 2600 msec. - $2700 > 2600 \Rightarrow$ **not absolutely valid.**

Ex 3: $R = \{\text{position, velocity, acceleration}\}$, $R_{rvi} = 100 \text{ msec}$, current time = 2600 msec. - Position (t=2500), Velocity (t=2550), Acceleration (t=2425) — each **absolutely valid**.
- Relative check: $|2550 - 2425| = 125 > 100 \Rightarrow$ **not relatively consistent.**

Point out: individual validity does not guarantee mutual (relative) consistency.

8. Concurrency Control in Real-Time Databases (Brief)

- Transactions access **several data items**; disk access makes them **long-duration**.
- For throughput, start a transaction **as soon as it is ready** (concurrently) rather than serially.
- Uncontrolled concurrency can **violate ACID** (e.g., one transaction's result overwritten by another).
- **Main goal:** ensure **non-interference** (isolation + atomicity).
- Achieved by enforcing **serializability** — concurrent execution equivalent to some **serial** execution.

9. Summary

- Real-time databases store large data **and** must respect **timing constraints**.
- They differ from traditional databases in **temporal data, timing constraints, and performance metric** (deadline misses vs. throughput).
- **Design issues (core):**
 - ① Disk-access delay $\Rightarrow$ missed deadlines
 - ② Protocol overhead $\Rightarrow$ unpredictable response time
 - ③ Cascading rollbacks $\Rightarrow$ unpredictable delay
- **Silver linings:** in-memory databases + simple, known, fixed transactions $\Rightarrow$ deterministic execution.
- **Temporal consistency** (absolute validity + relative consistency) keeps stored state true to the environment.

Conclusion

- The fundamental challenge of a real-time database is **predictability under timing constraints**, not just correctness.
- Good design **exploits the known, periodic, and simple nature** of real-time transactions and prefers **in-memory** storage.
- Correctness now has a **time dimension**: data must be **temporally consistent**, and transactions must **meet deadlines**.
***In one line:** A real-time database is not just a fast database — it is a database where being **late** is being **wrong**.*

References

- ① Rajib Mall, *Real-Time Systems: Theory and Practice*, Pearson Education India, 2007 — Chapter on Real-Time Databases.
- ② Standard RDBMS literature (as cited in the text) on relational databases, transactions, and ACID properties.